

지식그래프/온톨로지 개발자 (Backend / AI) 면접 완벽 가이드

비전공자를 위한 초기초부터 실전까지 — AnchorIQ 프로젝트 경험 기반

목차

- [Part 1: 온톨로지 & 지식그래프 기초](#)
- [Part 2: RDF / OWL / SPARQL \(시맨틱웹\)](#)
- [Part 3: Neo4j & Graph Database](#)
- [Part 4: AI / LLM / RAG](#)
- [Part 5: Java 백엔드 기본](#)
- [Part 6: 데이터 모델링 & DB 설계](#)
- [Part 7: AnchorIQ 프로젝트 기반 답변 전략](#)
- [Part 8: 실전 면접 시뮬레이션 50문 50답](#)

Part 1: 온톨로지 & 지식그래프 기초

1.1 온톨로지(Ontology)란?

초등학생도 이해할 수 있는 설명

온톨로지는 **"세상의 것들을 분류하고 관계를 정리하는 방법"**입니다.

예를 들어, 도서관에 가면:

- 책이 있고 (개체, Entity)
- 책에는 "소설", "과학", "역사" 같은 분류가 있고 (클래스, Class)
- 책과 저자 사이에 "~가 썼다"라는 관계가 있습니다 (관계, Relation)

이걸 컴퓨터가 이해할 수 있게 정리한 것이 온톨로지입니다.

정식 정의

온톨로지(Ontology)란 특정 도메인(분야)에 존재하는 **개념(Concept)**, **속성(Property)**, **관계(Relation)**를 명시적이고 형식적으로 정의한 것.

핵심 구성 요소

구성 요소	설명	예시 (해운 도메인)
Class (클래스)	개념의 분류/유형	선박(Vessel), 항구(Port), 회사(Company)
Instance (인스턴스)	클래스의 구체적 실체	"HMM 알헤시라스호", "부산항"
Property (속성)	인스턴스가 가진 특성	선박.무게 = 200,000톤
Relation (관계)	인스턴스 간 연결	"HMM 알헤시라스호" → [소유] → "HMM"
Axiom (공리)	반드시 성립하는 규칙	"모든 선박은 반드시 하나의 국적을 가진다"

면접에서 자주 나오는 질문

Q: 온톨로지와 데이터베이스 스키마의 차이는?

비교	DB 스키마	온톨로지
목적	데이터 저장	지식 표현 + 추론

관계 표현	FK(외래키)	다양한 시맨틱 관계
추론 가능?	불가능	가능 (OWL Reasoner)
유연성	스키마 변경 어려움	확장 용이
표준	SQL	RDF/OWL

A 예시: "DB 스키마는 데이터를 어떻게 저장할지 정의하는 것이고, 온톨로지는 데이터가 무엇을 의미하는지, 어떤 관계를 갖는지를 정의합니다. 가장 큰 차이는 ****추론(Reasoning)****이 가능하다는 것입니다. 예를 들어, 'A 회사가 B 선박을 소유하고, B 선박이 C 항구에 정박해 있다'면, 온톨로지에서는 'A 회사가 C 항구와 관련있다'를 자동으로 추론할 수 있습니다."

1.2 지식그래프(Knowledge Graph)란?

초등학생도 이해할 수 있는 설명

지식그래프는 ****지식을 점(노드)과 선(엣지)으로 연결한 지도****입니다.

삼성전자	--공급-->	Apple
Apple	--본사위치-->	쿠팡티노
삼성전자	--본사위치-->	수원
삼성전자	--업종-->	반도체

이렇게 연결하면, "삼성전자와 Apple의 관계는?" 이라는 질문에 답할 수 있습니다.

정식 정의

지식그래프(Knowledge Graph)란 **트리플(Triple)** 형태로 실세계 엔티티 간의 관계를 표현한 그래프 구조의 지식 베이스.

트리플(Triple)이란?

지식그래프의 최소 단위입니다:

(주어, 술어, 목적어) (Subject, Predicate, Object)
예시: (삼성전자, 공급한다, Apple) (HMM, 소유한다, 알헤시라스호) (부산항, 위치한다, 대한민국)

온톨로지 vs 지식그래프 차이

비교	온톨로지	지식그래프
성격	스키마 (설계도)	데이터 (실제 건물)
내용	클래스, 관계 정의	실제 인스턴스 + 관계
비유	"선박은 회사에 소속된다"는 규칙	"알헤시라스호는 HMM에 소속된다"는 사실
관계	온톨로지가 지식그래프의 뼈대	지식그래프가 온톨로지를 채운 것

핵심: 온톨로지 = 틀 (Schema), 지식그래프 = 틀에 데이터를 넣은 것

1.3 왜 지식그래프를 쓰는가?

관계형 DB로는 못하는 것

시나리오: "제재 대상 국가와 거래하는 회사가 소유한 선박 중 부산항에 입항한 것은?"

SQL로 풀면:

```
SELECT v.name
FROM vessels v
JOIN companies c ON v.company_id = c.id
JOIN company_country cc ON c.id = cc.company_id
JOIN countries co ON cc.country_id = co.id
JOIN sanctions s ON co.id = s.country_id
JOIN port_calls pc ON v.id = pc.vessel_id
JOIN ports p ON pc.port_id = p.id
WHERE p.name = '부산항' AND s.active = true;
-- 6개 테이블 JOIN... 느리고 복잡
```

지식그래프(Cypher)로 풀면:

```
MATCH (v:Vessel)-[:OWNED_BY]->(c:Company)-[:REGISTERED_IN]->(co:Country)-[:SANCTIONED_BY]->()
WHERE (v)-[:CALLED_AT]->(:Port {name: '부산항'})
RETURN v.name
-- 관계를 따라가기만 하면 됨. 직관적이고 빠름
```

지식그래프의 3가지 핵심 강점

1. 관계 탐색이 빠르다 — JOIN 없이 관계를 타고 이동 (O(1) per hop)
2. 유연하다 — 새로운 관계 추가가 스키마 변경 없이 가능
3. 추론이 가능하다 — "A→B→C"에서 "A→C" 관계를 자동 도출

Part 2: RDF / OWL / SPARQL (시맨틱웹)

2.1 시맨틱웹(Semantic Web)이란?

초등학생 설명

현재 웹은 사람이 읽는 웹입니다. 시맨틱웹은 컴퓨터도 의미를 이해하는 웹입니다.

현재 웹: <p>삼성전자는 반도체를 만든다</p> → 사람만 이해
시맨틱웹: (삼성전자, 생산한다, 반도체) → 컴퓨터도 이해

시맨틱웹 기술 스택 (피라미드)

```
[Trust]
[Proof]
[Logic / Rules]
[OWL (온톨로지 언어)]      ← 면접 출제
[RDFS (RDF 스키마)]
[RDF (데이터 모델)]        ← 면접 출제
[SPARQL (쿼리 언어)]       ← 면접 출제
[XML / JSON-LD (직렬화)]
[URI / IRI (식별자)]
```

2.2 RDF (Resource Description Framework)

한줄 정의

모든 지식을 **트리플(주어-술어-목적어)**로 표현하는 W3C 표준 데이터 모델

트리플 예시

```
<http://anchoriq.com/vessel/HMM-Algeciras>   ← 주어 (Subject)
<http://anchoriq.com/ontology/ownedBy>       ← 술어 (Predicate)
<http://anchoriq.com/company/HMM> .          ← 목적어 (Object)
```

사람이 읽으면: "HMM 알헤시라스호는 HMM이 소유한다"

RDF 직렬화 포맷 (저장 형식)

포맷	특징	예시
Turtle (.ttl)	가장 읽기 쉬움, 가장 많이 사용	ex:HMM ex:owns ex:Ship1 .
JSON-LD	JSON 기반, 웹 개발자 친화적	{"@id": "ex:HMM", "ex:owns": ...}
RDF/XML	XML 기반, 오래됨	<rdf:Description ...>
N-Triples	한 줄에 하나씩, 단순함	<s> <p> <o> .

면접 포인트

Q: RDF에서 URI를 쓰는 이유는? A: 전 세계적으로 유일한 식별자이기 때문입니다. 예를 들어, "삼성"이라는 이름은 삼성전자, 삼성물산 등 여러 개일 수 있지만, `http://example.com/company/samsung-electronics` 는 하나뿐입니다. 이를 통해 서로 다른 시스템의 데이터를 **연결(Linking)**할 수 있습니다.

2.3 RDFS (RDF Schema)

한줄 정의

RDF 위에 **클래스 계층구조와 속성 도메인/범위**를 정의하는 가벼운 스키마 언어

핵심 개념

```
# 클래스 정의
ex:Vessel rdf:type rdfs:Class .
ex:CargoShip rdfs:subClassOf ex:Vessel .    # 화물선은 선박의 하위 클래스

# 속성 정의
ex:ownedBy rdf:type rdf:Property .
ex:ownedBy rdfs:domain ex:Vessel .          # 주어는 선박
ex:ownedBy rdfs:range ex:Company .          # 목적어는 회사
```

RDFS로 가능한 추론

```
사실: ex:Algeciras rdf:type ex:CargoShip .
규칙: ex:CargoShip rdfs:subClassOf ex:Vessel .
추론: ex:Algeciras rdf:type ex:Vessel .    # 알헤시라스는 선박이다 (자동 추론!)
```

2.4 OWL (Web Ontology Language)

한줄 정의

RDFS보다 강력한 **논리적 표현력**을 가진 온톨로지 정의 언어 (W3C 표준)

RDFS vs OWL 차이

기능	RDFS	OWL
클래스 계층	O	O
속성 도메인/범위	O	O
동치 클래스 (equivalentClass)	X	O
역관계 (inverseOf)	X	O
추이적 관계 (transitiveProperty)	X	O
카디널리티 제약 (최소/최대 개수)	X	O
논리 연산 (union, intersection)	X	O

OWL 핵심 기능 5가지

1. 역관계 (Inverse Property)

```
ex:owns owl:inverseOf ex:ownedBy .
# "HMM이 알헤시라스를 소유한다" ↔ "알헤시라스는 HMM에 소유된다"
# 하나만 입력하면 반대도 자동 추론
```

2. 추이적 관계 (Transitive Property)

```
ex:locatedIn rdf:type owl:TransitiveProperty .
# 부산항이 부산에 위치, 부산이 한국에 위치
# → 부산항이 한국에 위치 (자동 추론!)
```

3. 동치 클래스 (Equivalent Class)

```
ex:FreightShip owl:equivalentClass ex:CargoShip .
# 화물선 = 화주선 (같은 의미)
```

4. 카디널리티 제약 (Cardinality Restriction)

```
ex:Vessel rdfs:subClassOf [
  rdf:type owl:Restriction ;
  owl:onProperty ex:hasFlag ;
  owl:cardinality 1      # 선박은 반드시 1개의 국적(깃발)을 가짐
] .
```

5. 분리합집합 (Disjoint Union)

```
ex:VesselStatus owl:oneOf (ex:Sailing ex:Anchored ex:Moored) .
# 선박 상태는 항해중/정박중/접안중 중 하나만 가능
```

OWL 서브언어 (표현력 순)

서브언어	표현력	추론 속도	용도
OWL Lite	낮음	빠름	간단한 분류
OWL DL	중간	보통	가장 많이 사용
OWL Full	높음	느림/불가	연구용

면접 팁: "저는 OWL DL을 사용할 것입니다. 추론이 결정 가능(Decidable)하면서 충분한 표현력을 제공하기 때문입니다."

2.5 SPARQL

한줄 정의

RDF 데이터를 조회하기 위한 그래프 쿼리 언어 (W3C 표준). SQL의 그래프 버전.

SQL vs SPARQL 비교

SQL	SPARQL
SELECT * FROM vessels	SELECT ?v WHERE { ?v rdf:type ex:Vessel }
WHERE v.name = 'HMM'	FILTER (?name = "HMM")
JOIN	트리플 패턴 매칭 (JOIN 불필요)
테이블 대상	그래프(트리플) 대상

SPARQL 기본 구조

```
PREFIX ex: <http://anchoriq.com/ontology/>

SELECT ?vesselName ?companyName
WHERE {
  ?vessel rdf:type ex:Vessel .           # 선박인 것 중에서
  ?vessel ex:name ?vesselName .         # 선박 이름 가져오고
  ?vessel ex:ownedBy ?company .         # 소유 회사 찾고
  ?company ex:name ?companyName .       # 회사 이름 가져옴
}
ORDER BY ?vesselName
LIMIT 10
```

SPARQL 쿼리 유형 4가지

유형	설명	예시
SELECT	변수 바인딩 반환 (SQL SELECT와 유사)	SELECT ?name WHERE {...}
CONSTRUCT	새로운 RDF 그래프 생성	CONSTRUCT { ?s ex:relatedTo ?o } WHERE {...}
ASK	참/거짓 반환	ASK { ex:HMM ex:owns ?ship }
DESCRIBE	리소스에 대한 모든 트리플 반환	DESCRIBE ex:HMM

SPARQL 고급 기능

1. OPTIONAL (LEFT JOIN과 유사)

```
SELECT ?vessel ?sanction
WHERE {
  ?vessel rdf:type ex:Vessel .
  OPTIONAL { ?vessel ex:sanctionedBy ?sanction } # 제재 없어도 결과 포함
}
```

2. FILTER (조건)

```
SELECT ?vessel ?score
WHERE {
  ?vessel ex:riskScore ?score .
  FILTER (?score > 80) # 리스크 80 이상만
}
```

3. UNION (OR 조건)

```
SELECT ?entity
WHERE {
  { ?entity rdf:type ex:Vessel }
  UNION
  { ?entity rdf:type ex:Company } # 선박 또는 회사
}
```

4. 서브쿼리

```
SELECT ?company (COUNT(?vessel) AS ?shipCount)
WHERE {
  ?vessel ex:ownedBy ?company .
}
GROUP BY ?company
HAVING (COUNT(?vessel) > 5) # 선박 5척 이상 보유 회사
```

SPARQL vs Cypher (Neo4j) 비교

비교	SPARQL	Cypher
대상	RDF 트리플 스토어	Property Graph (Neo4j)
표준	W3C 표준	Neo4j 독자
문법	트리플 패턴 매칭	ASCII 아트 패턴 ()-[]->()
추론	OWL Reasoner 연동 가능	기본 제공 안됨
직관성	학습 필요	상대적으로 직관적

면접 답변: "SPARQL은 W3C 표준으로 RDF 기반 트리플 스토어에서 사용하고, Cypher는 Neo4j의 Property Graph 모델에서 사용합니다. AnchorIQ 프로젝트에서는 Neo4j + Cypher를 사용했는데, 시맨틱웹 표준이 필요한 경우 RDF 스토어 + SPARQL로 전환할 수 있습니다. 두 접근 방식의 핵심 차이는 RDF가 추론(Reasoning)에 강하고, Property Graph가 성능과 직관성에서 강하다는 것입니다."

Part 3: Neo4j & Graph Database

3.1 그래프 데이터베이스란?

초등학생 설명

SNS를 생각하세요:

- 사람 = 노드(Node)
- 친구 관계 = 엣지(Edge)
- 이름, 나이 = 속성(Property)

이걸 저장하고 검색하는 DB가 그래프 데이터베이스입니다.

그래프 모델 2가지

모델	설명	대표 DB
Property Graph	노드/엣지에 속성(Key-Value) 부여 가능	Neo4j, Amazon Neptune
RDF Graph	트리플(주어-술어-목적어) 기반	GraphDB, Apache Jena

3.2 Neo4j 핵심 개념

데이터 모델 구성 요소

구성 요소	설명	예시
Node (노드)	개체	(:Vessel {name: "알헤시라스"})
Label (라벨)	노드의 유형	:Vessel, :Company, :Port
Relationship (관계)	노드 간 연결, 방향 있음	-[:OWNED_BY]->
Property (속성)	노드/관계의 Key-Value	{name: "HMM", founded: 1976}

Cypher 쿼리 언어 기초

노드 생성:

```
CREATE (v:Vessel {name: "알헤시라스", imo: "9863297", tonnage: 228283})
CREATE (c:Company {name: "HMM", country: "KR"})
```

관계 생성:

```
MATCH (v:Vessel {name: "알헤시라스"}), (c:Company {name: "HMM"})
CREATE (v)-[:OWNED_BY {since: 2020}]->(c)
```

조회:

```
// 기본 조회
MATCH (v:Vessel)-[:OWNED_BY]->(c:Company)
WHERE c.name = "HMM"
RETURN v.name, v.tonnage

// 2-hop: 회사의 국가까지
MATCH (v:Vessel)-[:OWNED_BY]->(c:Company)-[:REGISTERED_IN]->(co:Country)
RETURN v.name, c.name, co.name
```



```
// 3-hop: 국가의 제재 여부까지
MATCH (v:Vessel)-[:OWNED_BY]->(c:Company)-[:REGISTERED_IN]->(co:Country)-[:SANCTIONED_BY]->(s:Sanction)
RETURN v.name, s.reason

// 가변 깊이 탐색
MATCH path = (v:Vessel)-[*1..4]->(target) // 1~4 hop
WHERE v.name = "알헤시라스"
RETURN path
```

면접 빈출 질문

Q: Neo4j에서 인덱스는 어떻게 동작하나요?

A: "Neo4j는 두 가지 인덱스를 제공합니다:

- 1. **B-Tree 인덱스** — 속성 값으로 노드를 빠르게 찾을 때 (CREATE INDEX FOR (v:Vessel) ON (v.imo))
- 2. **Full-text 인덱스** — 텍스트 검색용 (Lucene 기반)

관계 탐색 자체는 인덱스가 필요 없습니다. Neo4j는 **index-free adjacency** 구조로, 각 노드가 인접 노드의 물리적 포인터를 직접 가지고 있어 관계 탐색이 O(1)입니다. 이것이 RDBMS에서 JOIN으로 관계를 탐색하는 것(O(log n))보다 빠른 이유입니다."

Q: Neo4j의 ACID 지원은?

A: "Neo4j는 완전한 ACID 트랜잭션을 지원합니다. 단일 노드/관계 생성뿐 아니라, 여러 노드와 관계를 하나의 트랜잭션으로 묶을 수 있습니다. AnchorIQ에서 선박-회사-국가 데이터를 한 번에 업데이트할 때 트랜잭션을 사용했습니다."

Q: Neo4j의 한계는?

A: "3가지 한계가 있습니다:

- 1. **대량 집계 연산(SUM, AVG)**은 RDBMS보다 느림 — 그래서 AnchorIQ에서 통계 데이터는 PostgreSQL에 저장
- 2. **샤딩 미지원 (CE 버전)** — Enterprise Edition에서만 Fabric으로 분산 가능
- 3. **전문 검색 한계** — 뉴스 본문 검색은 Elasticsearch가 더 적합해서 별도 운영"

3.3 Property Graph vs RDF Graph

비교	Property Graph (Neo4j)	RDF Graph (GraphDB 등)
데이터 모델	노드 + 라벨 + 속성 + 관계	트리플 (주어-술어-목적어)
쿼리 언어	Cypher	SPARQL
관계에 속성	O (관계도 속성 가질 수 있음)	X (Reification으로 우회)
표준화	ISO GQL (2024~)	W3C 표준 (성숙)
추론	별도 구현 필요	OWL Reasoner 내장
성능	높음 (네이티브 그래프 저장)	보통
적합한 용도	소셜 네트워크, 추천, 사기 탐지	데이터 통합, 시맨틱 검색, LOD

면접 답변: "프로젝트 요구사항에 따라 선택합니다. 시맨틱 추론과 데이터 통합이 핵심이면 RDF + SPARQL을, 성능과 직관적 모델링이 중요하면 Property Graph + Cypher를 선택합니다. 두 모델을 함께 쓸 수도 있습니다 — Neo4j의 Neosemantics(n10s) 플러그인으로 RDF 데이터를 Neo4j에 임포트할 수 있습니다."

Part 4: AI / LLM / RAG

4.1 LLM (Large Language Model) 기초

초등학생 설명

ChatGPT 같은 AI입니다. 엄청나게 많은 글을 읽고 학습해서, 질문에 대답할 수 있는 프로그램.

면접에서 알아야 할 핵심 개념

개념	설명	비유
Token	LLM이 처리하는 텍스트 단위 (약 4글자 = 1토큰)	글자 묶음
Context Window	한 번에 처리할 수 있는 토큰 수 (GPT-4: 128K)	작업대 크기
Prompt	AI에게 주는 입력/질문	주문서
Temperature	응답의 랜덤성 (0=정확, 1=창의적)	요리 양념 조절
Hallucination	AI가 사실이 아닌 것을 사실처럼 말하는 현상	최대 문제점
Fine-tuning	특정 도메인 데이터로 추가 학습	전문가 교육
Embedding	텍스트를 숫자 벡터로 변환	텍스트의 좌표

4.2 RAG (Retrieval-Augmented Generation)

이것이 이 공고의 핵심입니다

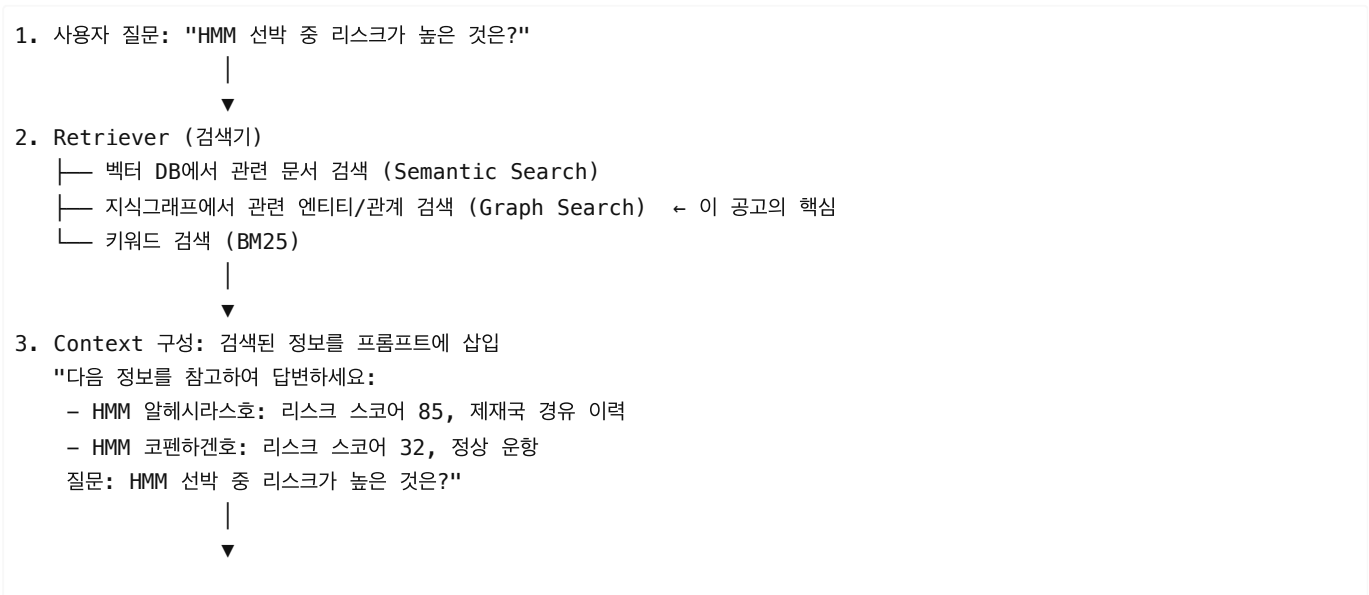
초등학생 설명

AI에게 질문할 때, 관련 자료를 먼저 찾아서 함께 주는 것.

시험을 볼 때:

- 일반 LLM = 기억만으로 시험 봄 → 기억이 틀릴 수 있음 (Hallucination)
- RAG = 오픈북 시험 → 책을 찾아보고 답함 → 정확도 높음

RAG 아키텍처



4. Generator (LLM)

"HMM 알헤시라스호가 리스크 스코어 85로 가장 높습니다.
제재국 경유 이력이 주요 원인입니다."

RAG의 핵심 구성 요소

구성 요소	역할	기술
Document Loader	원본 데이터 로드	PDF, DB, API 등에서 데이터 수집
Chunking	문서를 적절한 크기로 분할	고정 크기, 시맨틱 청킹 등
Embedding	텍스트를 벡터로 변환	OpenAI Ada, Sentence-BERT
Vector Store	벡터 저장/검색	Pinecone, Chroma, Weaviate, pgvector
Retriever	관련 문서 검색	Dense(벡터), Sparse(BM25), Hybrid
Prompt Template	검색 결과 + 질문 조합	LangChain PromptTemplate
LLM	최종 답변 생성	GPT-4, Claude 등

Knowledge Graph + RAG (GraphRAG) — 이 공고의 핵심

일반 RAG의 한계:

- 벡터 검색만으로는 **관계(Relation)**를 이해하지 못함
- "A와 B의 관계는?" 같은 질문에 약함

GraphRAG 해결:

사용자: "HMM과 제재국의 연결 관계를 알려줘"

1. 지식그래프에서 관계 탐색:
(HMM) - [:OWNS] -> (알헤시라스호) - [:VISITED] -> (이란 항구) - [:LOCATED_IN] -> (이란) - [:SANCTIONED]
2. 탐색 결과를 LLM 컨텍스트에 삽입
3. LLM이 관계를 자연어로 설명:
"HMM이 소유한 알헤시라스호가 이란 항구를 방문한 이력이 있으며, 이란은 현재 UN 제재 대상국입니다."

면접 빈출 질문

Q: RAG에서 Chunking 전략은 어떻게 세우나요?

A: "3가지를 고려합니다:

1. **Chunk 크기** — 너무 작으면 컨텍스트 부족, 너무 크면 노이즈 증가. 보통 512~1024 토큰.
2. **Overlap** — 청크 간 겹치는 부분 (50~100 토큰). 문맥이 끊기는 것 방지.
3. **전략 선택** — 고정 크기, 문단 기준, 시맨틱 기반 중 데이터 성격에 맞게 선택. 지식그래프와 결합하면, 청킹 단계에서 엔티티를 인식하고 해당 엔티티의 그래프 관계를 함께 가져올 수 있어 더 풍부한 컨텍스트를 제공합니다."

Q: RAG의 한계는?

A: "3가지 한계가 있습니다:

1. **검색 품질에 의존** — 잘못된 문서를 검색하면 답변도 틀림
2. **지연 시간** — 검색 단계가 추가되어 응답이 느려짐

3. **컨텍스트 윈도우 한계** — 검색 결과가 너무 많으면 다 넣을 수 없음 이를 보완하기 위해 Re-ranking, Query Decomposition 등의 기법을 사용합니다."

Q: 벡터 검색과 그래프 검색의 차이?

A: "벡터 검색은 **의미적 유사도** 기반으로 관련 문서를 찾고, 그래프 검색은 **구조적 관계**를 따라 연결된 정보를 찾습니다. 예를 들어, '삼성전자 협력사 리스크'를 검색하면:

- 벡터 검색: '삼성전자', '리스크'와 유사한 문서를 찾음
- 그래프 검색: 삼성전자 → 협력사 관계를 타고 → 각 협력사의 리스크를 수집 두 방식을 **하이브리드**로 결합하면 가장 좋은 결과를 얻을 수 있습니다."

4.3 LangChain & LlamaIndex

LangChain

개념	설명
Chain	LLM 호출을 연결하는 파이프라인
Agent	LLM이 도구를 선택해서 사용하는 자율 시스템
Tool	Agent가 사용할 수 있는 기능 (검색, 계산, API 호출 등)
Memory	대화 이력을 유지하는 메커니즘
Retriever	RAG에서 문서를 검색하는 컴포넌트

```
# LangChain + Neo4j RAG 예시 (개념 코드)
from langchain_community.graphs import Neo4jGraph
from langchain.chains import GraphCypherQAChain

graph = Neo4jGraph(url="bolt://localhost:7687", username="neo4j", password="password")

chain = GraphCypherQAChain.from_llm(
    llm=ChatOpenAI(model="gpt-4"),
    graph=graph,
    verbose=True
)

result = chain.run("HMM이 소유한 선박 중 리스크가 높은 것은?")
# 1. LLM이 자연어 → Cypher 쿼리 생성
# 2. Neo4j에서 실행
# 3. 결과를 LLM이 자연어로 변환
```

LlamaIndex

LangChain과 비슷하지만 **데이터 인덱싱과 검색에 특화**.

비교	LangChain	LlamaIndex
강점	다양한 체인/에이전트 구성	데이터 인덱싱/검색 최적화
적합	복잡한 워크플로우	RAG 중심 애플리케이션
그래프 지원	Neo4jGraph, GraphCypherQA	KnowledgeGraphIndex

Part 5: Java 백엔드 기본

5.1 면접에서 물어볼 Java 핵심

객체지향 프로그래밍 (OOP) 4대 원칙

원칙	한줄 정의	예시
캡슐화	데이터와 메서드를 하나로 묶고 외부 접근 제한	private 필드 + public 메서드
상속	부모 클래스의 속성/메서드를 자식이 물려받음	class CargoShip extends Vessel
다형성	같은 인터페이스로 다른 동작	vessel.calculateRisk() — 선종마다 다르게
추상화	핵심만 노출, 세부 구현 숨김	interface RiskCalculator

SOLID 원칙

원칙	설명	AnchorIQ 적용 예시
S - 단일 책임	클래스는 하나의 책임만	VesselService는 선박만, RiskService는 리스크만
O - 개방 폐쇄	확장에 열림, 수정에 닫힘	새 리스크 유형 추가 시 기존 코드 수정 없이 인터페이스 구현
L - 리스코프 치환	자식은 부모를 대체 가능	CargoShip을 Vessel 자리에 써도 동작
I - 인터페이스 분리	필요한 인터페이스만 의존	RiskCalculator, RiskReader 분리
D - 의존성 역전	추상화에 의존, 구현에 의존하지 않음	Service → Repository(인터페이스) ← RepositoryImpl

Spring Boot 핵심

Q: Spring의 IoC/DI란?

A: "IoC(제어의 역전)는 객체 생성과 생명주기 관리를 개발자가 아닌 Spring 컨테이너가 담당하는 것입니다. DI(의존성 주입)는 IoC를 구현하는 방법으로, 객체가 필요로 하는 의존성을 외부에서 주입받는 것입니다.

예를 들어, RiskService 가 VesselRepository 를 필요로 할 때, 직접 new VesselRepositoryImpl() 을 하지 않고, 생성자에 VesselRepository 인터페이스를 선언하면 Spring이 자동으로 구현체를 주입합니다."

```
// DI 예시
@Service
public class RiskServiceImpl implements RiskService {
    private final VesselRepository vesselRepository; // 인터페이스에 의존

    public RiskServiceImpl(VesselRepository vesselRepository) { // Spring이 주입
        this.vesselRepository = vesselRepository;
    }
}
```

Q: Spring Data JPA란?

A: "JPA(Java Persistence API)는 Java 객체를 DB 테이블에 매핑하는 ORM 표준이고, Spring Data JPA는 이를 더 편하게 사용할 수 있게 해주는 Spring 모듈입니다. 인터페이스만 정의하면 기본적인 CRUD 쿼리를 자동 생성합니다."

Q: @Transactional의 동작 원리는?

A: "Spring AOP 프록시가 메서드 실행 전에 트랜잭션을 시작하고, 정상 완료 시 커밋, 예외 발생 시 롤백합니다. 주의점:

- 1. 같은 클래스 내부 호출은 프록시를 거치지 않아 트랜잭션이 안 걸림
- 2. 외부 API 호출은 트랜잭션 안에 넣지 않음 — 외부 API가 느리면 DB 커넥션을 오래 점유
- 3. AnchorIQ에서는 Tier 1(돈/유저)만 @Transactional, Tier 2(온톨로지)는 Kafka 최종 일관성 패턴 사용"

REST API 설계

원칙	설명	좋은 예	나쁜 예
자원 중심	URL은 동사가 아닌 명사	/vessels/{id}	/getVessel
HTTP 메서드 활용	CRUD를 메서드로 구분	GET /vessels	POST /getVessels
복수형 사용	컬렉션은 복수	/vessels	/vessel
상태 코드 활용	결과를 코드로 표현	201 Created	항상 200 OK

5.2 DDD (Domain-Driven Design) 기본

핵심 개념

개념	설명	AnchorIQ 예시
Entity	고유 식별자가 있는 객체. 비즈니스 로직 보유	Vessel (IMO 번호로 식별)
Value Object	식별자 없이 값으로 동등성 판단	RiskScore, Coordinate
Aggregate Root	관련 Entity 묶음의 진입점	Vessel이 PortCall들을 관리
Repository	Aggregate 저장/조회 인터페이스	VesselRepository
Domain Service	여러 Aggregate에 걸친 비즈니스 로직	SupplyChainRiskService
Application Service	오케스트레이션만, 비즈니스 로직 없음	VesselApplicationService

면접 팁: "Entity에 비즈니스 로직을 넣어 풍부한 도메인 모델을 만들었습니다. Application Service는 순서만 조율하고, 실제 판단은 Entity가 합니다."

Part 6: 데이터 모델링 & DB 설계

6.1 관계형 DB 설계 기본

정규화

정규형	한줄 규칙	위반 예시
1NF	모든 컬럼은 원자값	전화번호: "010-1234, 010-5678" → 분리
2NF	부분 함수 종속 제거	복합키의 일부에만 종속 → 분리
3NF	이행 함수 종속 제거	A→B→C에서 A→C 직접 종속 → 분리

인덱스

Q: 인덱스는 왜 빠른가요?

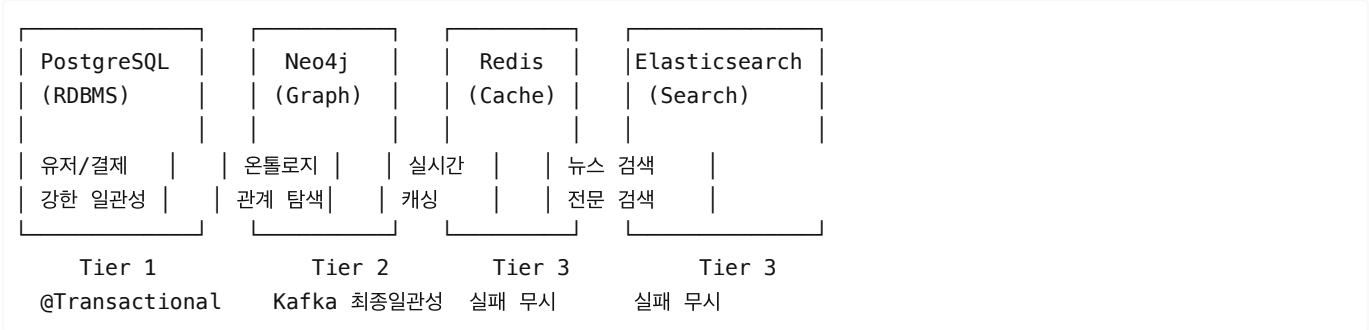
A: "B-Tree 구조로 저장되어 이진 탐색이 가능합니다. 100만 건에서 약 20번의 비교만으로 원하는 데이터를 찾을 수 있습니다 (log₂(1,000,000) ≈ 20). 인덱스 없이는 100만 건을 순차 탐색해야 합니다."

Q: 인덱스의 단점은?

A: "쓰기 성능이 떨어집니다. INSERT/UPDATE 시 인덱스도 함께 갱신해야 하고, 디스크 공간도 추가로 사용합니다. 읽기가 잦은 컬럼에만 선별적으로 생성합니다."

6.2 다중 DB 전략

AnchorIQ의 4개 DB 전략 (면접 어필 포인트)



면접 답변: "각 DB의 강점에 맞게 데이터를 분산 저장했습니다:

- **PostgreSQL** — ACID가 필요한 금융 데이터 (결제, 구독)
- **Neo4j** — 관계 탐색이 핵심인 온톨로지 데이터
- **Redis** — 밀리초 단위 응답이 필요한 실시간 캐시
- **Elasticsearch** — 뉴스 본문 전문 검색

트랜잭션은 3-Tier로 분류하여, 돈이 오가는 Tier 1만 강한 일관성을 보장하고, 나머지는 최종 일관성으로 성능을 확보했습니다."

Part 7: AnchorIQ 프로젝트 기반 답변 전략

7.1 자기소개 스크립트

"저는 AnchorIQ라는 해운 공급망 리스크 탐지 플랫폼을 개발했습니다.

11개의 외부 API에서 수집한 선박, 항구, 기업, 제재, 뉴스 데이터를 **Neo4j 지식그래프로 융합**하여, 선박-기업-국가-제재 간의 관계를 추론합니다.

예를 들어, A 기업이 소유한 B 선박이 제재 대상국 C의 항구를 방문했다면, 이를 **4-hop 그래프 탐색**으로 자동 탐지하고, AI가 리스크를 평가하여 알림을 발송합니다.

이 경험을 통해 **온톨로지 설계, 그래프 DB 최적화, AI 연동, 이벤트 기반 데이터 파이프라인**을 직접 구축한 경험이 있습니다."

7.2 공고 요구사항별 답변 매핑

"도메인 분석 기반 데이터 모델링 및 온톨로지 설계"

답변: "해운 도메인을 분석하여 10개 이상의 노드 유형(Vessel, Port, Company, Country, Sanction, Route, Weather, EEZ, News, Risk)과 15개 이상의 관계 유형을 정의했습니다. 도메인 전문가 인터뷰 없이 공개 데이터와 IMO 규정을 분석하여 온톨로지를 설계했으며, DDD의 Aggregate Root 패턴으로 데이터 일관성을 보장했습니다."

"Knowledge Graph 구축"

답변: "Neo4j를 사용하여 25척의 선박, 6개의 해운 회사, 주요 항구와 국가 간의 지식그래프를 구축했습니다. 4-hop 깊이까지의 관계 탐색을 지원하며, 자주 조회되는 경로는 Redis에 캐싱하여 응답 시간을 최적화했습니다."

"Java 기반 백엔드 서비스 개발"

답변: "Spring Boot 3.4 기반 멀티모듈 프로젝트로, DDD 레이어 구조(Controller → Application Service → Domain Service → Repository)를 적용했습니다. 5개 모듈(core, api, collector, ai, automation)을 Gradle로 관리하며, 모듈 간 의존성 규칙을 엄격히 적용했습니다. core 모듈은 순수 도메

인으로 외부 의존성이 없습니다."

"REST API 기반 데이터 서비스 개발"

답변: "174개의 REST 엔드포인트와 3개의 WebSocket 엔드포인트를 설계/구현했습니다. Pageable 기반 페이지네이션, 통일된 에러 응답 포맷, JWT 인증을 적용했습니다."

"LLM 기반 AI 서비스 연계 개발"

답변: "OpenAI 게이트웨이를 통해 LLM과 연동하여, 지식그래프의 데이터를 컨텍스트로 제공하고 리스크를 판단하는 시스템을 구축했습니다. 이는 GraphRAG 패턴과 유사한 구조로, 그래프에서 관련 엔티티/관계를 검색한 후 LLM에 전달하여 자연어 분석 결과를 생성합니다."

"데이터 통합 및 메타데이터 관리"

답변: "11개의 서로 다른 외부 API(AIS, 항구 정보, 날씨, 뉴스, UN 제재 등)에서 수집한 이질적 데이터를 Kafka 이벤트 파이프라인으로 통합했습니다. 각 데이터 소스의 포맷이 달라 정규화 과정을 거쳐 온톨로지 스키마에 맞게 변환한 후 Neo4j에 저장합니다."

7.3 기술적 깊이를 보여주는 질문 대응

"Neo4j에서 성능 문제는 어떻게 해결했나요?"

답변: "3가지 전략을 적용했습니다:

1. **4-hop 제한** — 무한 탐색 방지. 대부분의 유의미한 관계는 4-hop 이내에서 발견됩니다.
2. **Redis 캐싱** — 자주 조회되는 그래프 경로를 Redis에 캐싱하여 반복 조회 최적화.
3. **라벨 인덱스** — `CREATE INDEX FOR (v:Vessel) ON (v.imo)` 등 자주 검색되는 속성에 인덱스 생성.

추가로, Neo4j의 EXPLAIN/PROFILE 명령으로 쿼리 실행 계획을 분석하고 최적화했습니다."

"Kafka를 왜 사용했나요?"

답변: "3가지 이유입니다:

1. **데이터 소스 간 디커플링** — 수집기와 저장소가 독립적으로 스케일 가능
2. **최종 일관성 보장** — Neo4j 쓰기 실패 시 Kafka에 이벤트가 남아있어 재처리 가능
3. **다중 Consumer** — 하나의 이벤트를 Redis 캐시 갱신, Neo4j 저장, AI 분석이 동시에 소비

Dead Letter Topic(DLT)으로 실패 이벤트를 별도 관리하고, 3회 재시도 후 DLT로 이동하는 정책을 적용했습니다."

Part 8: 실전 면접 시뮬레이션 50문 50답

카테고리 1: 온톨로지 & 지식그래프 (15문)

Q1. 온톨로지란 무엇인가요?

A: "특정 도메인의 개념, 속성, 관계를 명시적으로 정의한 것입니다. 쉽게 말하면 '이 분야의 것들을 어떻게 분류하고 연결할지'를 정한 설계도입니다. 데이터베이스 스키마와 다른 점은, 온톨로지는 추론(Reasoning)이 가능하다는 것입니다."

Q2. 지식그래프란 무엇이고, 왜 필요한가요?

A: "실세계 엔티티 간의 관계를 그래프(노드+엣지) 구조로 표현한 지식 베이스입니다. 필요한 이유는 관계형 DB에서는 다중 JOIN이 필요한 복잡한 관계 탐색을 그래프에서는 직관적이고 빠르게 할 수 있기 때문입니다. AnchorIQ에서 선박→회사→국가→제재 4단계 관계를 한 번의 쿼리로 탐색합니다."

Q3. 온톨로지와 지식그래프의 관계는?

A: "온톨로지는 지식그래프의 스키마(뼈대)입니다. '선박은 회사에 소속된다'는 규칙이 온톨로지이고, '알헤시라스호는 HMM에 소속된다'는 사실이 지식그래프에 저장됩니다. 온톨로지 없이 지식그래프를 만들면 일관성 없는 데이터가 됩니다."

Q4. 온톨로지 설계 경험을 구체적으로 설명해주세요.

A: "해운 도메인을 분석하여 10개 노드 유형과 15개 관계 유형을 정의했습니다. 먼저 핵심 개념(Vessel, Company, Port, Country)을 식별하고, 이들 간의 관계(OWNED_BY, REGISTERED_IN, CALLED_AT, LOCATED_IN)를 정의했습니다. 그 다음 제재(SANCTIONED_BY), 날씨(AFFECTED_BY), 경제 구역(WITHIN_EEZ) 등 도메인 특화 관계를 추가했습니다."

Q5. 지식그래프에서 '추론(Reasoning)'이란?

A: "명시적으로 저장하지 않은 새로운 사실을 기존 데이터와 규칙으로부터 도출하는 것입니다. 예를 들어, 'A 회사가 B 선박을 소유' + 'B 선박이 C 항구 방문' → 'A 회사가 C 항구와 관련있다'를 자동 추론합니다. OWL에서는 TransitiveProperty, InverseOf 등의 규칙으로 이를 지원합니다."

Q6. Property Graph와 RDF Graph의 차이는?

A: "Property Graph는 노드와 엣지에 속성(Key-Value)을 직접 붙일 수 있고, Neo4j + Cypher가 대표적입니다. RDF Graph는 트리플(주어-술어-목적어) 기반이고, GraphDB + SPARQL이 대표적입니다. 핵심 차이는 RDF가 W3C 표준이라 데이터 통합에 유리하고 OWL 추론이 가능하지만, Property Graph가 성능과 직관성에서 앞섭니다."

Q7. 온톨로지 설계 시 가장 중요한 원칙은?

A: "3가지입니다:

- 모듈화** — 도메인별로 온톨로지를 분리하여 독립적으로 관리 (해운, 리스크, 금융)
- 재사용** — 기존 표준 어휘(Schema.org, Dublin Core)를 최대한 활용
- 확장성** — 새로운 개념과 관계를 기존 구조 변경 없이 추가할 수 있어야 함"

Q8. 온톨로지의 품질을 어떻게 평가하나요?

A: "4가지 기준으로 평가합니다:

- 완전성(Completeness)** — 도메인의 주요 개념이 모두 표현되었는가
- 일관성(Consistency)** — 논리적 모순이 없는가 (OWL Reasoner로 검증)
- 간결성(Conciseness)** — 불필요한 중복 없이 최소한으로 표현했는가
- 적용성(Applicability)** — 실제 쿼리와 추론에 잘 활용되는가"

Q9. 지식그래프의 데이터 품질은 어떻게 관리하나요?

A: "3단계로 관리합니다:

- 입력 시** — 스키마 검증 (필수 속성 체크, 데이터 타입 검증)
- 저장 후** — 일관성 검증 (고아 노드, 끊어진 관계 탐지)
- 주기적** — 외부 소스와 대조하여 최신성 확인 AnchorIQ에서는 Kafka Consumer에서 데이터 정규화 및 검증을 수행한 후 Neo4j에 저장합니다."

Q10. 지식그래프에 새로운 도메인을 추가할 때 접근 방식은?

A: "4단계로 접근합니다:

- 도메인 분석** — 핵심 개념과 관계 식별
- 기존 온톨로지 매핑** — 새 도메인이 기존 노드와 어떻게 연결되는지 정의
- 스키마 확장** — 새 노드 유형과 관계 유형 추가 (기존 스키마 수정 최소화)
- 데이터 적재** — ETL 파이프라인으로 데이터 변환 후 그래프에 적재"

Q11. 지식그래프의 확장성(Scalability) 문제를 어떻게 해결하나요?

A: "3가지 전략을 씁니다:

- 쿼리 최적화** — 탐색 깊이 제한 (4-hop 이내), 인덱스 적용
- 캐싱** — 자주 조회되는 서브그래프를 Redis에 캐싱

Q12. 트리플(Triple)이란?

A: "지식그래프의 최소 단위로, (주어, 술어, 목적어) 형태입니다. 예: (HMM, 소유한다, 알헤시라스호). RDF에서는 모든 지식을 이 트리플로 표현하며, 트리플들이 모여 그래프를 형성합니다."

Q13. Linked Open Data(LOD)란?

A: "웹에 공개된 데이터를 RDF로 표현하고 URI로 식별하여 서로 연결한 것입니다. 4단계 원칙이 있습니다:

1. URI로 식별
 2. HTTP URI 사용 (접근 가능)
 3. RDF/SPARQL 표준 사용
 4. 다른 URI에 링크 DBpedia, Wikidata가 대표적입니다."
-

Q14. 온톨로지 개발 도구는 어떤 것이 있나요?

A: "Protégé가 가장 널리 쓰이는 오픈소스 온톨로지 편집기입니다. OWL 온톨로지를 GUI로 설계하고, 내장된 Reasoner(HermiT, Pellet)로 일관성을 검증할 수 있습니다. 상용으로는 TopBraid Composer가 있습니다."

Q15. 이 회사에서 지식그래프를 어떻게 활용할 수 있을까요?

A: "공고의 'LLM 기반 AI 서비스 연계'를 보면, Knowledge Graph + RAG 구조가 핵심일 것입니다. 도메인 데이터를 지식그래프로 구축하고, 사용자 질문에 대해 그래프에서 관련 정보를 검색한 후 LLM에 컨텍스트로 제공하는 GraphRAG 파이프라인을 구현하는 것이 주요 업무일 것으로 예상합니다."

카테고리 2: RDF / OWL / SPARQL (10문)

Q16. RDF란 무엇인가요?

A: "Resource Description Framework의 약자로, W3C가 정한 데이터 모델 표준입니다. 모든 지식을 (주어, 술어, 목적어) 트리플로 표현합니다. 각 요소는 URI로 고유하게 식별되며, 이를 통해 웹 상의 서로 다른 데이터를 연결할 수 있습니다."

Q17. OWL이 RDFS보다 나은 점은?

A: "RDFS는 클래스 계층과 속성 도메인/범위만 정의할 수 있지만, OWL은 역관계(inverseOf), 추이적 관계(transitiveProperty), 카디널리티 제약, 논리 연산(union, intersection)을 추가로 지원합니다. 즉, 더 정밀한 규칙 정의와 추론이 가능합니다."

Q18. SPARQL 쿼리를 작성해본 적이 있나요?

A: "Neo4j의 Cypher를 주로 사용했고, SPARQL은 학습 수준입니다. 다만 두 언어 모두 그래프 패턴 매칭 기반이라 개념이 유사합니다. Cypher의 MATCH (a)-[:REL]->(b) 가 SPARQL의 ?a ex:REL ?b 에 대응됩니다. 필요시 빠르게 전환할 수 있습니다."

Q19. RDF 직렬화 포맷의 차이는?

A: "Turtle이 가장 읽기 쉽고 널리 쓰입니다. JSON-LD는 웹 개발자에게 친숙한 JSON 기반입니다. RDF/XML은 오래된 형식이고, N-Triples는 한 줄에 한 트리플로 파싱이 가장 단순합니다. 시스템 간 데이터 교환에는 JSON-LD, 사람이 읽어야 하면 Turtle을 선택합니다."

Q20. OWL Reasoner란?

A: "온톨로지의 규칙을 기반으로 새로운 사실을 자동 추론하는 엔진입니다. 예를 들어, 'A는 B의 하위 클래스' + 'x는 A 타입'이면 'x는 B 타입'을 추론합니다. HermiT, Pellet, FaCT++가 대표적이며, Protégé에 내장되어 온톨로지 일관성 검증에도 사용됩니다."

Q21. SPARQL의 CONSTRUCT 쿼리는 언제 사용하나요?

A: "기본 그래프에서 새로운 RDF 그래프를 생성할 때 사용합니다. 예를 들어, 여러 소스의 데이터를 통합하여 하나의 통일된 그래프를 만들거나, 추론 결과를 별도 그래프로 저장할 때 유용합니다."

Q22. Named Graph란?

A: "RDF 트리플에 네 번째 요소(그래프 이름)를 추가한 것입니다. 트리플의 출처나 맥락을 구분할 수 있습니다. 예: (HMM, owns, Ship1, [graph:kiwoom-api](#)) — 이 사실의 출처가 키움 API라는 것을 표현. 데이터 출처 관리와 접근 제어에 활용됩니다."

Q23. SHACL과 OWL의 차이는?

A: "OWL은 추론을 위한 것이고, SHACL(Shapes Constraint Language)은 데이터 검증을 위한 것입니다. OWL로 '선박은 회사에 소속될 수 있다'는 규칙을 정의하고, SHACL로 '선박은 반드시 IMO 번호가 있어야 한다'는 제약 조건을 검증합니다."

Q24. RDF에서 Blank Node란?

A: "URI가 없는 익명 노드입니다. 복잡한 구조를 표현할 때 중간 노드로 사용합니다. 예: '선박이 좌표(위도, 경도)를 가진다'를 표현할 때, 좌표 자체는 URI가 필요 없는 Blank Node가 됩니다. 다만, 남용하면 데이터 연결이 어려워지므로 최소화하는 것이 좋습니다."

Q25. Ontology Alignment(온톨로지 정렬)이란?

A: "서로 다른 온톨로지 간의 동일한 개념을 매핑하는 작업입니다. 예: A 시스템의 '화물선(CargoShip)'과 B 시스템의 '화물운반선(FreightVessel)'이 같은 개념임을 매핑합니다. OWL의 owl:equivalentClass, owl:sameAs를 사용하며, 이질적 데이터 소스 통합에 필수적입니다."

카테고리 3: AI / RAG / LLM (10문)

Q26. RAG란 무엇이고 왜 필요한가요?

A: "Retrieval-Augmented Generation의 약자로, LLM이 답변을 생성하기 전에 관련 정보를 먼저 검색하여 컨텍스트로 제공하는 기법입니다. 필요한 이유는 LLM의 Hallucination(환각) 문제를 해결하기 위해서입니다. LLM이 학습하지 못한 최신 정보나 도메인 특화 정보를 검색 결과로 보완합니다."

Q27. Knowledge Graph + RAG (GraphRAG)의 장점은?

A: "일반 RAG는 벡터 유사도로 문서를 검색하지만, 관계(Relation)를 이해하지 못합니다. GraphRAG는 지식그래프의 구조적 관계를 활용하여:

1. 엔티티 간 경로를 찾아 맥락을 풍부하게 제공
 2. 다단계 추론이 필요한 질문에 답변 가능
 3. 정확한 사실 기반 응답 (그래프 데이터가 근거) AnchorIQ에서 이와 유사한 패턴을 적용했습니다."
-

Q28. Embedding이란?

A: "텍스트를 고차원 숫자 벡터로 변환하는 것입니다. '왕' - '남자' + '여자' = '여왕'처럼 의미적 연산이 가능합니다. RAG에서는 문서와 질문을 Embedding 하여 벡터 공간에서 유사한 문서를 검색합니다. OpenAI의 text-embedding-ada-002가 많이 사용됩니다."

Q29. Vector DB란?

A: "Embedding 벡터를 저장하고, 유사도 기반 검색(Approximate Nearest Neighbor)을 수행하는 특화 데이터베이스입니다. Pinecone, Chroma, Weaviate, Milvus가 대표적이며, PostgreSQL의 pgvector 확장을 쓸 수도 있습니다. RAG의 Retriever 역할을 합니다."

Q30. Chunking 전략은 어떻게 세우나요?

A: "데이터 성격에 따라 다릅니다:

- 고정 크기 — 단순하지만 문맥이 끊길 수 있음 (512~1024 토큰)
- 문단/섹션 기준 — 구조화된 문서에 적합

- **시맨틱** — 의미 단위로 분할 (Embedding 유사도 기반) Overlap(50~100 토큰)을 두어 문맥 손실을 방지하고, 너무 크면 노이즈, 너무 작으면 컨텍스트 부족이므로 실험으로 최적값을 찾습니다."

Q31. LangChain이란?

A: "LLM 기반 애플리케이션 개발을 위한 프레임워크입니다. Chain(LLM 호출 연결), Agent(자율적 도구 사용), Retriever(RAG 검색), Memory(대화 이력) 등의 컴포넌트를 제공합니다. Neo4j와의 연동도 지원하여 GraphCypherQChain으로 자연어 질문을 Cypher 쿼리로 변환할 수 있습니다."

Q32. Fine-tuning vs RAG, 언제 무엇을 쓰나요?

A: "Fine-tuning은 모델의 행동 방식을 바꿀 때(특정 도메인 용어, 응답 스타일), RAG는 최신 정보나 사실 기반 답변이 필요할 때 사용합니다. 대부분의 경우 **RAG**를 먼저 시도하는 것이 좋습니다. Fine-tuning은 비용이 높고, 데이터 변경 시 재학습이 필요하기 때문입니다."

Q33. Hallucination을 줄이는 방법은?

A: "5가지 방법이 있습니다:

1. **RAG** — 사실 기반 컨텍스트 제공
2. **프롬프트 엔지니어링** — '모르면 모른다고 답하세요' 지시
3. **Temperature 낮추기** — 0에 가깝게 설정
4. **출처 명시** — 답변에 근거 데이터 표시 강제
5. **사실 검증 파이프라인** — 그래프 DB 데이터와 대조 검증"

Q34. Prompt Engineering의 핵심 기법은?

A: "주요 기법 3가지:

1. **Few-shot** — 예시를 함께 제공 ('이런 입력에는 이런 출력')
2. **Chain-of-Thought** — 단계별 사고를 유도 ('단계적으로 생각하세요')
3. **System Prompt** — 역할과 규칙을 명시 ('당신은 해운 리스크 분석 전문가입니다') 지식그래프와 결합할 때는 그래프 데이터를 구조화된 형태로 프롬프트에 삽입하는 것이 중요합니다."

Q35. AI Agent란?

A: "LLM이 스스로 도구(Tool)를 선택하여 작업을 수행하는 자율 시스템입니다. 예를 들어, '서울에서 부산까지 배송 리스크를 알려줘'라는 질문에 Agent가:

1. 지식그래프에서 경로 검색 (Tool 1)
2. 날씨 API 조회 (Tool 2)
3. 리스크 계산 (Tool 3) 을 자동으로 순서대로 실행합니다. LangChain의 Agent 프레임워크로 구현할 수 있습니다."

카테고리 4: Java 백엔드 (10문)

Q36. Spring Framework의 핵심 특징 3가지는?

A: "1. **IoC/DI** — 객체 생성과 의존성 관리를 컨테이너가 담당 2. **AOP** — 로깅, 트랜잭션 등 횡단 관심사를 분리 3. **PSA(Portable Service Abstraction)** — JPA, JMS 등 다양한 기술에 대한 일관된 추상화 제공"

Q37. JPA의 N+1 문제란?

A: "연관 엔티티를 조회할 때, 목록 1번 + 각 항목의 연관 엔티티 N번 = 총 N+1번의 쿼리가 실행되는 문제입니다. 해결 방법:

1. **Fetch Join** — `@Query('SELECT v FROM Vessel v JOIN FETCH v.company')`
2. **@EntityGraph** — 선언적으로 함께 로딩할 엔티티 지정
3. **Batch Size** — `@BatchSize(size=100)` 으로 IN 절 묶기"

Q38. 인터페이스 기반 설계를 하는 이유는?

A: "3가지 이유입니다:

1. **DIP(의존성 역전)** — 상위 모듈이 하위 모듈의 구현이 아닌 추상화에 의존
2. **테스트 용이성** — Mock 객체로 쉽게 대체 가능
3. **확장성** — 구현체 교체가 기존 코드 수정 없이 가능 AnchorIQ에서 모든 Service, Repository, Gateway를 인터페이스 기반으로 설계했습니다."

Q39. Gradle 멀티 모듈의 장점은?

A: "1. **의존성 경계 강제** — core 모듈이 외부 프레임워크에 의존하지 않도록 빌드 레벨에서 보장 2. **빌드 성능** — 변경된 모듈만 재빌드 3. **MSA 전환 용이** — 필요시 모듈을 독립 서비스로 분리 가능 AnchorIQ에서 5개 모듈을 이 구조로 관리했습니다."

Q40. Kafka Consumer에서 메시지 처리 실패 시 어떻게 하나요?

A: "3단계 전략입니다:

1. **즉시 재시도** — 1초 간격 3회
2. **DLT(Dead Letter Topic) 이동** — 3회 실패 시 별도 토픽으로 이동
3. **수동 재처리** — DLT에 쌓인 메시지를 분석 후 원인 해결 뒤 재처리 이를 통해 하나의 실패가 전체 파이프라인을 멈추지 않도록 합니다."

Q41. Virtual Thread (Java 21)란?

A: "기존 OS Thread보다 훨씬 가벼운 JVM 레벨 경량 스레드입니다. 수백만 개를 생성해도 메모리 부담이 적습니다. 블로킹 I/O(DB 조회, API 호출)에서 기존 스레드는 대기 중 리소스를 점유하지만, Virtual Thread는 대기 중 자동으로 양보하여 효율적입니다. AnchorIQ에서 WebClient + Virtual Thread 조합으로 외부 API 호출을 처리했습니다."

Q42. 보상 트랜잭션(Compensation Transaction)이란?

A: "분산 시스템에서 원본 트랜잭션이 실패했을 때 이전에 성공한 작업을 되돌리는 트랜잭션입니다. 예: 결제 성공 → 재고 차감 실패 → 결제 취소(보상 트랜잭션) 실행. AnchorIQ에서는 외부 API 호출을 @Transactional 밖에서 수행하고, 실패 시 보상 트랜잭션으로 원복합니다."

Q43. Redis를 캐시로 쓸 때 주의점은?

A: "3가지입니다:

1. **TTL 필수** — 만료 시간 없으면 메모리 부족 발생
2. **캐시 일관성** — DB 데이터 변경 시 캐시도 무효화 (Cache Aside 패턴)
3. **캐시 스템퍼드** — TTL 동시 만료 시 DB에 부하 집중 → Jitter 추가"

Q44. REST API에서 인증을 어떻게 처리하나요?

A: "JWT(JSON Web Token) 기반입니다. 로그인 시 Access Token(단기)과 Refresh Token(장기)을 발급합니다. Access Token은 HttpOnly Cookie에 저장하여 XSS 공격을 방지하고, 만료 시 Refresh Token으로 자동 갱신합니다. AnchorIQ에서 Access Token 5시간, Refresh Token 7일로 설정했습니다."

Q45. @Transactional에서 주의할 점은?

A: "3가지입니다:

1. **같은 클래스 내부 호출 시 프록시가 동작하지 않음** — 별도 클래스로 분리 필요
2. **checked exception은 기본적으로 롤백 안됨** — rollbackFor 로 명시
3. **외부 API를 트랜잭션 안에서 호출하면 안됨** — DB 커넥션을 오래 점유하여 성능 저하"

카테고리 5: 인성/문화적합성 (5문)

Q46. 왜 지식그래프/온톨로지 분야에 관심을 갖게 되었나요?

A: "AnchorIQ 프로젝트에서 해운 데이터를 관계형 DB로만 처리하려 했을 때 한계를 느꼈습니다. 선박-회사-국가-제재 간의 복잡한 관계를 다중 JOIN으로 탐색하는 것이 비효율적이어서 Neo4j 그래프 DB를 도입했고, 그 과정에서 지식그래프의 강력함을 체감했습니다. 데이터 간의 '관계'를 통해 새로운 인사이트를 발견하는 것에 큰 매력을 느꼈습니다."

Q47. 비전공자인데 기술적으로 어떻게 학습하나요?

A: "프로젝트 기반 학습을 합니다. AnchorIQ를 만들면서 Neo4j, Kafka, DDD 등을 필요에 의해 학습했고, 공식 문서를 먼저 읽고 직접 구현하면서 익힙니다. 문제가 생기면 docs/ 폴더에 트러블슈팅 기록을 남겨 같은 실수를 반복하지 않습니다. 이론보다 실습이 저에게 효과적이라고 느낍니다."

Q48. 새로운 기술을 빠르게 학습한 경험은?

A: "AnchorIQ에서 Neo4j와 Kafka를 처음 사용했습니다. Neo4j는 그래프 모델링 개념부터 Cypher 쿼리, 4-hop 최적화까지 프로젝트를 진행하면서 학습했고, Kafka는 프로듀서/컨슈머 설계부터 DLT 처리까지 직접 구현했습니다. 두 기술 모두 2주 내에 프로젝트에 적용할 수준까지 올렸습니다."

Q49. 팀에서 의견 충돌이 있을 때 어떻게 하나요?

A: "데이터와 근거로 대화합니다. '이 방법이 좋다'가 아니라 '이 방법의 벤치마크 결과가 이렇다'로 설득합니다. AnchorIQ 설계 시에도 모든 기술 선택에 '왜 이것을 선택했는가' 근거를 문서화했습니다. 의견이 다르면 빠르게 PoC(Proof of Concept)를 만들어 검증합니다."

Q50. 지원하면서 이 회사에 기여할 수 있는 점은?

A: "3가지입니다:

- 지식그래프 실전 경험 — Neo4j 기반 온톨로지 설계부터 4-hop 탐색, 캐싱 최적화까지 구현한 경험
- AI 연동 경험 — LLM과 지식그래프를 결합한 GraphRAG 유사 패턴 구현
- 데이터 파이프라인 경험 — 11개 이질적 데이터 소스를 Kafka 기반으로 통합하여 지식그래프에 적재

특히 '도메인 분석 → 온톨로지 설계 → 지식그래프 구축 → AI 연계'의 전체 흐름을 직접 경험한 것이 강점입니다."

부록 A: 핵심 용어 사전

용어	설명
Triple	(주어, 술어, 목적어) — 지식 표현의 최소 단위
URI/IRI	자원의 고유 식별자 (URL의 상위 개념)
Ontology	도메인 개념과 관계의 형식적 정의
Knowledge Graph	트리플 기반 그래프 구조의 지식 베이스
RDF	트리플 기반 데이터 모델 표준 (W3C)
RDFS	RDF 위의 가벼운 스키마 언어
OWL	논리적 추론이 가능한 온톨로지 언어
SPARQL	RDF 그래프 쿼리 언어
Cypher	Neo4j의 그래프 쿼리 언어
Property Graph	노드/엣지에 속성을 부여하는 그래프 모델
Reasoner	온톨로지 규칙 기반 자동 추론 엔진
RAG	검색 증강 생성 — LLM + 외부 검색

GraphRAG	지식그래프를 검색 소스로 사용하는 RAG
Embedding	텍스트를 숫자 벡터로 변환
Vector DB	벡터 유사도 검색 특화 DB
Chunking	문서를 적절한 크기로 분할
Hallucination	AI가 사실이 아닌 것을 생성하는 현상
Fine-tuning	특정 데이터로 LLM 추가 학습
LangChain	LLM 기반 앱 개발 프레임워크
LlamaIndex	데이터 인덱싱/검색 특화 LLM 프레임워크
DDD	도메인 주도 설계
Aggregate Root	관련 Entity 묶음의 진입점
ACID	원자성, 일관성, 격리성, 지속성
DLT	Dead Letter Topic — 실패 메시지 저장
LOD	Linked Open Data — 연결된 공개 데이터
SHACL	RDF 데이터 검증 언어

부록 B: 면접 전 체크리스트

반드시 준비할 것

- ☐ AnchorIQ 아키텍처 그림 (화이트보드 설명용)
- ☐ Neo4j 데모 (선박-회사-제재 관계 탐색 시연)
- ☐ 온톨로지 설계 문서 (노드 유형, 관계 유형, 설계 근거)
- ☐ GitHub 레포지토리 정리
- ☐ RAG 개념 코드 (LangChain + Neo4j 간단 예시)

보강 학습 추천 (우선순위 순)

- SPARQL 기본 쿼리** — SELECT, FILTER, OPTIONAL 직접 작성해보기
- RDF Turtle 문법** — 간단한 온톨로지를 Turtle로 작성해보기
- Protégé** — 설치 후 간단한 온톨로지 만들어보기
- LangChain + Neo4j** — GraphCypherQChain 예제 실행해보기
- OWL 추론** — Protégé의 Reasoner로 추론 결과 확인해보기

마지막 한마디: 비전공자라는 것은 약점이 아닙니다. AnchorIQ처럼 "직접 만들어본 경험"이 있다는 것이 가장 강력한 증거입니다. 면접에서는 이론을 외운 것이 아니라, 왜 그 기술을 선택했고 어떤 문제를 해결했는지를 말하세요.